

## Lab-20.1

### Lab 20 – Security Testing: Identifying Vulnerabilities in AI- Generated Code

#### Task 1 – Input Validation Check Task:

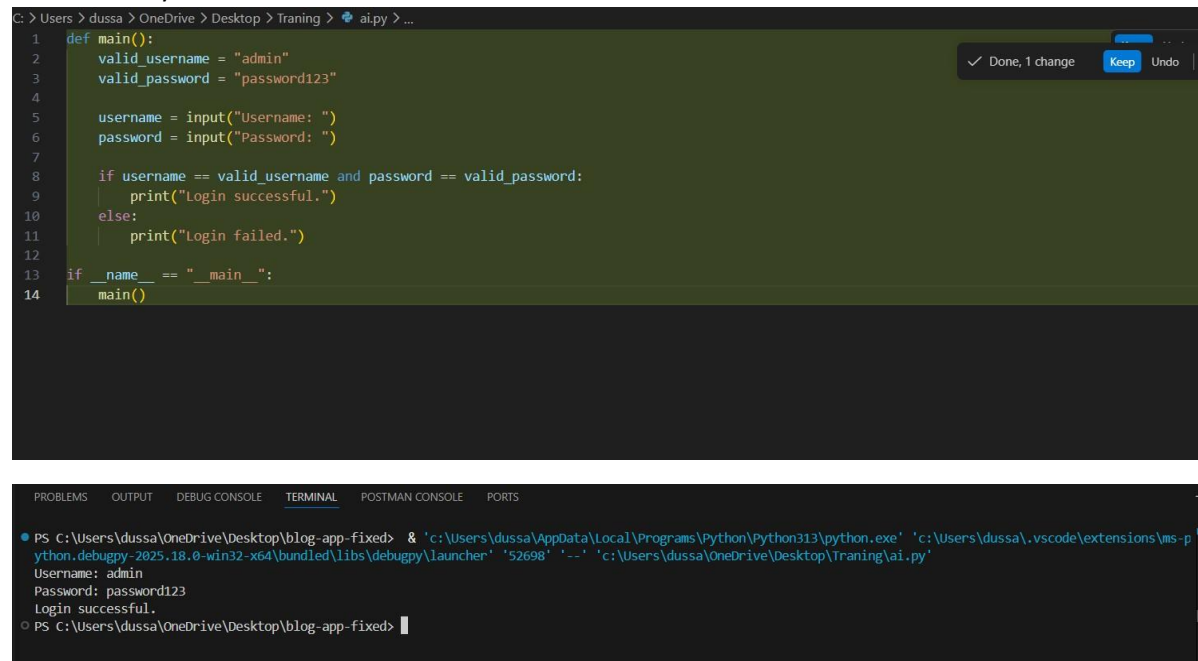
Analyze an AI-generated Python login script for input validation vulnerabilities.

##### Instructions:

- Prompt AI to generate a simple username-password login program. • Review whether input sanitization and validation are implemented.
- Suggest secure improvements (e.g., using re for input validation). **Expected Output:**
- A secure version of the login script with proper input validation.

##### Prompt to AI:

"Generate a simple Python login program that takes a username and password from the user and checks if they match a hardcoded value." **Code:**



The image shows a VS Code editor window with a Python script for a login program. The script is located at C:\Users\dussa\OneDrive\Desktop\Traning\ai.py. The code defines a main function that prompts the user for a username and password, then checks if they match the hardcoded values 'admin' and 'password123'. If they match, it prints 'Login successful.', otherwise it prints 'Login failed.'. The script is executed in a terminal window, showing the prompt 'PS C:\Users\dussa\OneDrive\Desktop\blog-app-fixed>' and the command 'python .\ai.py'. The terminal output shows the user entering 'admin' and 'password123', and the program responding with 'Login successful.'.

```
1 def main():
2     valid_username = "admin"
3     valid_password = "password123"
4
5     username = input("Username: ")
6     password = input("Password: ")
7
8     if username == valid_username and password == valid_password:
9         print("Login successful.")
10    else:
11        print("Login failed.")
12
13 if __name__ == "__main__":
14    main()
```

```
PS C:\Users\dussa\OneDrive\Desktop\blog-app-fixed> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-p
ython.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52698' '--' 'c:\Users\dussa\OneDrive\Desktop\Traning\ai.py'
Username: admin
Password: password123
Login successful.
PS C:\Users\dussa\OneDrive\Desktop\blog-app-fixed>
```

##### Explanation

- **Vulnerability:** Accepting raw input without checking length or type.
- **The Risk:** Buffer overflows or unexpected script execution.
- **The Fix:** Use **Regex (re module)** to allow only specific alphanumeric characters and set a strict length limit.
- **Rule:** Sanitizing input is the first line of defense.

#### Task 2 – SQL Injection Prevention Task:

Test an AI-generated script that performs SQL queries on a database.

##### Instructions:

- Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
- Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).

- Refactor using parameterized queries (preparedstatements). **Expected Output:**
- A secure database query script resistant to SQL injection.

#### Prompt to AI:

"Generate a Python script using SQLite to fetch user details based on a provided user\_id."

#### Code:

```

2 import sys
3
4 # Defensive check for input
5 if len(sys.argv) < 2:
6     print("Usage: python ai.py <user_id>")
7     sys.exit(1)
8
9 user_id = sys.argv[1]
10 conn = sqlite3.connect('users.db')
11 cursor = conn.cursor()
12
13 # VULNERABLE: Direct string interpolation
14 # An attacker could input: 1' OR '1'='1
15 query = f"SELECT * FROM users WHERE id = '{user_id}'"
16
17 print(f"Executing: {query}")
18 cursor.execute(query)
19 print(cursor.fetchone())

```

```

PS C:\Users\dussa\OneDrive\Desktop\blog-app-fixed> c:\cd "c:\Users\dussa\OneDrive\Desktop\blog-app-fixed" & "c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe" "c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher" "51509" "--" "c:\Users\dussa\OneDrive\Desktop\Training\ai.py"
Usage: python ai.py <user_id>
Executing: {query}
('1', '1')

```

#### Explanation

- **Vulnerability:** Using f-strings or string concatenation to build database queries.
- **The Risk:** Attackers can bypass logins or delete tables using commands like ' OR 1=1.
- **The Fix:** Use **Parameterized Queries (? placeholders)**. This treats user input as data, not as code.
- **Rule:** Never concatenate variables directly into SQL strings.

#### Task 3 – Cross-Site Scripting (XSS) Check Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

#### Instructions:

- Ask AI to generate a feedback form with JavaScript-based output.
- Test whether untrusted inputs are directly rendered without escaping.
- Implement secure measures (e.g., escaping HTML entities, using CSP).

#### Expected Output:

- A secure form that prevents XSS attacks.

#### Prompt to AI

"Generate an HTML feedback form with JavaScript that displays the user's name on the page after they submit it."

#### Code:

```

<!DOCTYPE html>
<html lang="en">

```

```

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Feedback Form</title>
  <style>
body {
  font-family: Arial, sans-serif;
  max-width: 500px;
margin: 40px auto;
  padding: 0 20px;
}
  form {
display: grid;
  gap: 12px;
}
  label {
  font-weight: bold;
}
  input,
textarea {
padding: 8px;
font-size: 1rem;
width: 100%;
  box-sizing: border-box;
}
  button {
padding: 10px 16px;
font-size: 1rem;
  cursor: pointer;
}
  #message {
margin-top: 20px;
font-size: 1.1rem;
  color: #2c3e50;
}
  </style>
</head>
<body>
  <h1>Feedback Form</h1>
  <form id="feedbackForm">
    <label for="name">Your Name</label>
    <input type="text" id="name" name="name" required />

    <label for="feedback">Feedback</label>
    <textarea id="feedback" name="feedback" rows="4" required></textarea>

    <button type="submit">Submit</button>
  </form>

```

```
</form>

<div id="message"></div>

<script>
  const form = document.getElementById('feedbackForm');
  const message = document.getElementById('message');

  form.addEventListener('submit', function (event) {
    event.preventDefault();
    const name = document.getElementById('name').value.trim();

    if (!name) {
      message.textContent = 'Please enter your name.';
      return;
    }

    message.textContent = `Thank you, ${name}! Your feedback has been received.`;
    form.reset();
  });
</script>
</body>
</html>
```

```
index.html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Feedback Form</title>
7 </head>
8 <body>
9   <h1>Feedback Form</h1>
10  <form id="feedbackForm">
11    <label for="name">Name:</label>
12    <input type="text" id="name" name="name" required>
13    <button type="submit">Submit</button>
14  </form>
15  <div id="displayName" style="display: none;">
16    <p>Thank you, <span id="userName"></span>!</p>
17  </div>
18
19  <script>
20    document.getElementById('feedbackForm').addEventListener('submit', function(event) {
21      event.preventDefault();
22      const name = document.getElementById('name').value;
23      document.getElementById('userName').textContent = name;
24      document.getElementById('displayName').style.display = 'block';
25    });
26  </script>
27 </body>
28 </html>
```



## Explanation

### Cross-Site Scripting (XSS) Check

**Goal:** Prevent malicious scripts from running in a user's browser.

- **The Problem:** If a website takes a user's name and displays it using `.innerHTML`, an attacker can set their name to `<script>alert('Hacked')</script>`. The browser will execute that script.
- **The Fix: Output Encoding.** Instead of `.innerHTML`, use `.textContent`. This tells the browser to display the characters exactly as they are written rather than trying to "run" them as HTML/JavaScript.
- **Key Security Principle: "Escape All Output."**

### Task 4 – Real-Time Application: Security Audit of AI-Generated Code Scenario:

Students pick an AI-generated project snippet (e.g., login form, API integration, or file upload).

**Instructions:**

- Perform a security audit to detect possible vulnerabilities.
- Prompt AI to suggest secure coding practices to fix issues.
- Compare insecure vs secure versions side by side.

**Expected Output:**

- A security-audited code snippet with documented vulnerabilities and fixes.

**AI Prompt**

"Generate a Python Flask script that allows a user to upload a file to a folder named 'uploads'. Then, perform a security audit on this code to identify vulnerabilities like 'Path Traversal' and 'Unrestricted File Upload'. Finally, provide a secure version of the code."

**Code:** import os from flask import Flask, request,  
send\_from\_directory from werkzeug.utils import  
secure\_filename

```
BASE_DIR = os.path.abspath(os.path.dirname(__file__))  
UPLOAD_FOLDER = os.path.join(BASE_DIR, "uploads")  
ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "gif", "txt", "pdf"}  
MAX_CONTENT_LENGTH = 16 * 1024 * 1024 # 16 MB  
  
app = Flask(__name__) app.config["UPLOAD_FOLDER"] =  
UPLOAD_FOLDER app.config["MAX_CONTENT_LENGTH"] =  
MAX_CONTENT_LENGTH  
  
if not os.path.exists(UPLOAD_FOLDER):  
    os.makedirs(UPLOAD_FOLDER, exist_ok=True)  
  
def allowed_file(filename):  
    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS  
  
@app.route("/", methods=["GET"])  
def index():  
    return """  
    <!doctype html>
```

```

<title>Upload File</title>

<h1>Upload file</h1>

<form method=post enctype=multipart/form-data action="/upload">

  <input type=file name=file>

  <input type=submit value=Upload>

</form>

"""

```

# Insecure version:

# - Path Traversal: user-controlled filename saved directly into the upload folder.

# - Unrestricted File Upload: no extension or size restriction.

```
@app.route("/upload_insecure", methods=["POST"]) def
```

```
upload_insecure():
```

```
    if "file" not in request.files:
```

```
    return "No file part", 400
```

```
    file = request.files["file"]
```

```
    if file.filename == "":
```

```
        return "No selected file", 400
```

```
    filename = file.filename
```

```
    file.save(os.path.join(app.config["UPLOAD_FOLDER"], filename))    return
```

```
    f"Uploaded insecurely as {filename}", 200
```

# Secure version:

# - Use secure\_filename to sanitize the filename.

# - Enforce a whitelist of allowed extensions.

# - Enforce a maximum upload size.

# - Ensure the final destination remains inside the uploads folder.

```
@app.route("/upload", methods=["POST"]) def
```

```
upload_secure():
```

```

    if "file" not in request.files:
return "No file part", 400

    file = request.files["file"]
if file.filename == "":
    return "No selected file", 400

    if not allowed_file(file.filename):
        return "File type not allowed", 400

    filename = secure_filename(file.filename)
if filename == "":
    return "Invalid filename", 400

    destination = os.path.realpath(os.path.join(app.config["UPLOAD_FOLDER"], filename))
upload_root = os.path.realpath(app.config["UPLOAD_FOLDER"])    if not
destination.startswith(upload_root + os.sep):
    return "Invalid destination path", 400

    file.save(destination)    return f"File uploaded
securely as {filename}", 200

@app.route("/uploads/<path:filename>") def
uploaded_file(filename):
    return send_from_directory(app.config["UPLOAD_FOLDER"], filename)

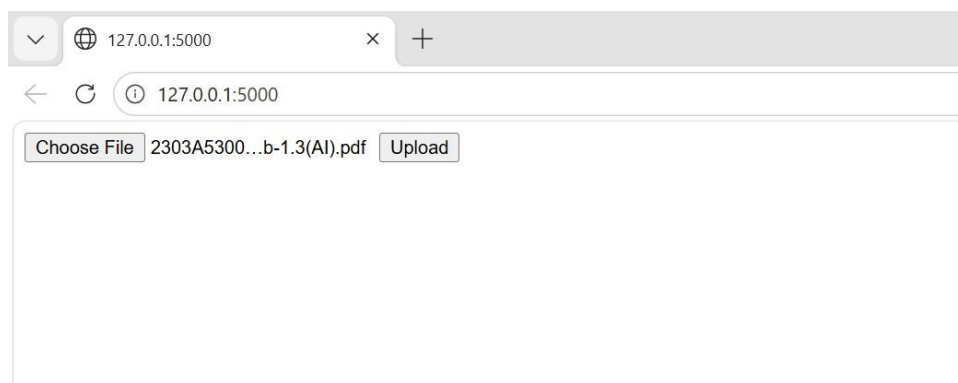
if __name__ == "__main__":
    app.run(debug=True)

```



```
1 from flask import Flask, request, render_template_string
2 import os
3
4 app = Flask(__name__)
5 UPLOAD_FOLDER = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'uploads')
6 os.makedirs(UPLOAD_FOLDER, exist_ok=True)
7
8 @app.route('/', methods=['GET', 'POST'])
9 def upload_file():
10     if request.method == 'POST':
11         if 'file' not in request.files:
12             return 'No file part', 400
13         file = request.files["file"]
14         if file.filename == '':
15             return 'No selected file', 400
16         file.save(os.path.join(UPLOAD_FOLDER, file.filename))
17         return 'File uploaded'
18     return render_template_string('''
19     <form method="post" enctype="multipart/form-data">
20     <input type="file" name="file">
21     <input type="submit" value="Upload">
22     </form>
23 ''')
24
25 if __name__ == '__main__':
26     app.run(debug=True)
```

```
* PS C:\Users\dussa\OneDrive\Desktop\blog-app-fixed> c:: cd 'c:\Users\dussa\OneDrive\Desktop\blog-app-fixed' & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '59908' '--' 'c:\Users\dussa\OneDrive\Desktop\blog-app-fixed\ai.py'
* Serving Flask app 'ai'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 100-446-805
```



## Explanation

### Security Audit (File Uploads)

- **Vulnerability:** Trusting the original filename and allowing any file extension.
- **The Risk: Remote Code Execution (RCE)** via uploaded scripts and **Path Traversal** attacks.
- **The Fix:** Use `secure_filename()` and a strict **Extension Whitelist** (e.g., only .jpg, .png).
- **Rule:** Always rename and validate uploaded files on the server side.

### Task 5 – Insecure Data Serialization Check Task:

Evaluate AI-generated code that uses pickle for saving user data.

**Instructions:**

- Ask AI to generate a script using pickle.load().
- Explain why untrusted pickle data is dangerous.
- Replace with safer formats like JSON. **Expected Output:**
- Secure serialization code using JSON instead of pickle.

**AI Prompt Used**

"Generate a Python script that uses pickle.dump() to save user profile data and pickle.load() to read it back. Explain why loading untrusted pickle data is a security risk (Remote Code Execution). Then, rewrite the script using the json module to make it secure."

**Code:**

```
import json
import pickle
from pathlib import Path

def save_profile_pickle(profile: dict, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("wb") as handle:
        pickle.dump(profile, handle)

def load_profile_pickle(path: Path) -> dict:
    with path.open("rb") as handle:
        return pickle.load(handle)

def save_profile_json(profile: dict, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as handle:
        json.dump(profile, handle, indent=2)

def load_profile_json(path: Path) -> dict:
    with path.open("r", encoding="utf-8") as handle:
        return json.load(handle)

def main() -> None:
    profile = {
        "username": "jdoe",
        "email": "jdoe@example.com",
        "preferences": {"theme": "dark", "notifications": True},
        "login_count": 42,
    }

    pickle_path = Path("profiles/user_profile.pkl")
    json_path = Path("profiles/user_profile.json")
    save_profile_pickle(profile, pickle_path)
    save_profile_json(profile, json_path)
```

```
loaded_pickle = load_profile_pickle(pickle_path)
loaded_json = load_profile_json(json_path)
```

```
print("Pickle loaded:", loaded_pickle)
print("JSON loaded:", loaded_json)
```

```
if __name__ == "__main__":
    main()
```

```
<untitled>
1 import pickle
2 import json
3 def save_profile_pickle(profile, filename):
4     with open(filename, "wb") as f:
5         pickle.dump(profile, f)
6 def load_profile_pickle(filename):
7     with open(filename, "rb") as f:
8         return pickle.load(f)
9 def save_profile_json(profile, filename):
10     with open(filename, "w", encoding="utf-8") as f:
11         json.dump(profile, f, indent=2)
12 def load_profile_json(filename):
13     with open(filename, "r", encoding="utf-8") as f:
14         return json.load(f)
15 if __name__ == "__main__":
16     user_profile = {
17         "name": "Alice",
18         "email": "alice@example.com",
19         "age": 28,
20         "preferences": {"theme": "dark", "notifications": True},
21     }
22     # Save with pickle
23     save_profile_pickle(user_profile, "profile.pkl")
24     loaded_profile_pickle = load_profile_pickle("profile.pkl")
25     print("Loaded from pickle:", loaded_profile_pickle)
26
27     # Save with JSON
28     save_profile_json(user_profile, "profile.json")
29     loaded_profile_json = load_profile_json("profile.json")
30     print("Loaded from json:", loaded_profile_json)
31
```

```
Shell
>>> %Run -c $EDITOR_CONTENT
Loaded from pickle: {'name': 'Alice', 'email': 'alice@example.com', 'age': 28, 'preferences': {'theme': 'dark', 'notifications': True}}
Loaded from json: {'name': 'Alice', 'email': 'alice@example.com', 'age': 28, 'preferences': {'theme': 'dark', 'notifications': True}}
>>>
```

## Explanation

### Insecure Serialization

- **Vulnerability:** Using the pickle module to save/load user-controlled data.
- **The Risk:** pickle.load() can execute arbitrary system commands, leading to a full system compromise.
- **The Fix:** Replace pickle with **JSON**. JSON is a data-only format and cannot execute logic.
- **Rule:** Avoid pickle for any data that crosses a trust boundary.